Model Collapse in an In-Context Learning Transformer

The base and extended tasks, methodological revisions,
and a controlled comparison of model-generated against real data

Author names

October 5, 2026

Abstract

Model collapse is the degradation of models trained on data produced by earlier models. We study it in a setting where the true data distribution is known exactly: a two-layer, attention-only transformer that learns a symbolic in-context learning task. In the base task the model outputs one label, fully determined by the context, so recursive training can only introduce errors. We extend the task so that the model also predicts the next symbol, a uniformly random choice between the symbols in the context, and that symbol's label. The extended output therefore carries a real probability distribution, which recursive training can distort.

A first five-generation run of the extended pipeline turned up four problems: the planned collapse metric was blind to collapse by construction, the generation-0 model memorised its training pool, generation was confounded with the size of the training pool, and there was only one seed. We describe the revised pipeline (per-sequence metrics computed from the model's probabilities, checkpoint selection on held-out pool data, a matched real-data control, and three paired seeds) and report a controlled comparison.

Because real data is never removed from the training pool, nothing runs away. Within the model-generated chain, the KL divergence of the next-symbol distribution from the truth moves only from 0.425 to 0.389 nats over four recursive generations. The matched real-data control reaches 0.157 nats, so the model-generated chain falls further behind in all three seeds, with the gap growing every generation. Model-generated data preserves the generation-0 model's errors, and each later model learns them as if they were part of the task. The base pipeline, run with the same seeds, keeps its query-label accuracy at 0.999 after four recursive generations, and its model-written labels agree with the truth in at least 99.7% of sequences in every generation.

Contents

1 Introduction **3**

2 Background **3**
2.1 In-context learning and induction heads 3
2.2 Model collapse 3

3 Tasks **4**
3.1 Base task 4
3.2 Extended task 4
3.3 Analytic reference values 5

4 Model **5**
4.1 Architecture 5
4.2 Training objective 6
4.3 Generating continuations 6

5 Recursive training pipeline **6**

6 Measuring collapse **7**
6.1 Accuracy and loss per token . . . 7
6.2 The next-symbol distribution . . . 7
6.3 Self-consistency and type validity . . . 8
6.4 Statistics of the generated data . . . 8
6.5 Why the emitted-symbol histogram cannot detect collapse . . . 8

7 Methodological revisions **9**
7.1 Why rigour matters for this project . . . 9
7.2 Revision 1: measure collapse where it can happen . . . 9
7.3 Revision 2: stop training before the model memorises its pool . . . 10
7.4 Revision 3: a matched real-data control . . . 10
7.5 Revision 4: replicate across seeds . . . 11
7.6 Smaller corrections . . . 11

8 Experimental setup **12**

9 Results **13**
9.1 Sanity checks . . . 13
9.2 Model-generated against real data . . . 13
9.3 Paired differences . . . 13
9.4 What the model-generated data contains . . . 15
9.5 The view without a control . . . 15
9.6 Base pipeline . . . 15

10 Discussion **16**
10.1 What the results show . . . 16
10.2 Mechanism . . . 17
10.3 Base against extended . . . 17
10.4 The role of checkpoint selection . . . 17
10.5 Limitations . . . 17
10.6 Future work . . . 17

11 Conclusion **18**

A Repository map **18**

B Summary CSV columns (extended pipeline) **18**

1 Introduction

Generative models are increasingly trained on data that other models produced. Shumailov et al. [1] showed that when each generation of a model is trained on the previous generation's output, models progressively lose the tails of the original distribution, a process they called *model collapse*. How quickly that happens depends on what the training data keeps: Gerstgrasser et al. [2] found that *accumulating* synthetic data alongside the original real data, rather than *replacing* it, keeps the error bounded.

This project studies collapse in a deliberately small setting where the true data distribution is known exactly: an attention-only transformer trained on a symbolic in-context learning (ICL) task of the kind used to study induction heads [3, 4]. Each sequence presents symbol–label pairs whose labels are drawn afresh for every sequence, so the model can only succeed by reading the mapping from the context.

Two pipelines are compared:

- In the **base** pipeline the model predicts one token, the label of a queried symbol, which the context fully determines.
- In the **extended** pipeline the model also predicts which symbol comes next, a uniformly random choice between the context symbols, and that symbol's label. The output therefore carries a genuine distribution, which recursive training can distort in ways that accuracy does not show.

This document covers:

- the tasks, model and recursive pipeline in full (Sections 3–5);
- how collapse is measured, including why an intuitive metric fails in this task (Section 6);
- the four methodological problems found in the first full run, the evidence for each, and how the pipeline was revised (Section 7);
- a replicated, paired comparison of model-generated against real data, together with the base pipeline run with the same seeds (Sections 8–9);
- what the results support and what they do not (Section 10).

2 Background

2.1 In-context learning and induction heads

An induction head is an attention circuit that, on seeing a token, looks back to an earlier occurrence of the same token and copies whatever followed it. Olsson et al. [3] identified such heads as a mechanism behind in-context learning, and showed that two-layer attention-only transformers are enough to form them. In the task used here, answering the query means finding the context position that holds the query symbol and copying the label that follows it, exactly the operation an induction head performs. Singh et al. [4] studied the formation of these circuits in this kind of setup (two context pairs, five labels drawn afresh per sequence), and the architecture below follows their defaults.

2.2 Model collapse

In the recursive setting, generation i trains a model M_i on data that includes samples from M_{i-1}. Any error or bias in M_{i-1} enters the training targets of M_i, which may learn it as if it were real structure. Shumailov et al. [1] describe two compounding sources: statistical error

from finite sampling, which loses rare events first, and functional error from the model's limited expressiveness. Gerstgrasser et al. [2] contrast two regimes. When each generation *replaces* the previous data, the error grows with the number of generations; when generations *accumulate* on top of the original real data, the error stays bounded. The pipeline in this project, following the assignment, accumulates.

3 Tasks

Throughout, $N = 50$ is the number of symbols, $L = 5$ the number of labels and $K = 2$ the number of symbol–label pairs in the context.

3.1 Base task

For each sequence:

1. draw K distinct symbols $c_1, \ldots, c_K$ uniformly without replacement from $\{0, \ldots, N-1\}$;
2. give each one a label $\ell(c_j)$ drawn independently and uniformly from $\{0, \ldots, L-1\}$; the assignment is consistent within the sequence but drawn afresh for every sequence;
3. draw a query index q uniformly from $\{1, \ldots, K\}$.

The input is $(c_1, \ell(c_1), \ldots, c_K, \ell(c_K), c_q)$, of length $2K+1 = 5$, and the target is $y_q = \ell(c_q)$. Every position is a one-hot vector of dimension $N + L = 55$: symbol c sets index c and label l sets index $N + l$. Because labels are re-drawn every sequence, no global symbol-to-label map exists, and the only way to answer is to read the pairing from the context. For example, with $K = 2$:

$$(\mathrm{A}, 3, \mathrm{B}, 1, \mathrm{A}) \mapsto 3, \qquad (\mathrm{A}, 0, \mathrm{C}, 4, \mathrm{A}) \mapsto 0.$$

3.2 Extended task

The extended task keeps the same context and query and asks for a three-token continuation $(y_q, x_{\text{next}}, y_{\text{next}})$:

- $y_q = \ell(c_q)$, the query's label, exactly as in the base task;
- $x_{\text{next}} = c_n$ with n drawn uniformly from $\{1, \ldots, K\}$, independently of q (so x_{next} may repeat the query symbol);
- $y_{\text{next}} = \ell(x_{\text{next}})$, the label of the chosen symbol.

The full sequence has length $2K + 4 = 8$:

position	0	1	2	3	4	5	6	7
token	c_1	$\ell(c_1)$	c_2	$\ell(c_2)$	c_q	y_q	x_{next}	y_{next}
predicted from						4	5	6

Inputs and targets share one joint vocabulary of size $V = N + L = 55$, the same layout as the one-hot inputs, so the output layer is the dual of the input encoding.

The middle token is the point of the extension. Which symbol comes next is a fair coin flip between the two context symbols, so the model's output at that position should be a distribution, not a single answer. Including the query symbol among the candidates matters: with $K = 2$, excluding it would leave exactly one candidate, x_{next} would be deterministic, and there would be no distribution for recursive training to distort. The third token adds a dependency: y_{next} is correct only if the model binds the symbol it just chose to that symbol's label in the context.

Table 1: True-distribution reference values for $K = 2$, $N = 50$, $L = 5$. The first block is what collapse of the next-symbol choice moves; the last row is a property of the task that no per-sequence rule can change.

Quantity	Value	Derivation
Next-symbol choice entropy (per sequence)	$\log K = 0.693$ nats	uniform over K context symbols
Probability on the preferred context symbol	$1/K = 0.50$	as above
Probability outside the context	0	$x_{\text{next}} \in \{c_1, \ldots, c_K\}$
KL from the true next-symbol distribution	0	definition
Cross-entropy floor at x_{next}	$\log K = 0.693$ nats	entropy of the true choice
Accuracy ceiling at x_{next}	$1/K = 0.50$	any argmax hits half the time
Floor of the mean loss over three tokens	$\log K/3 = 0.231$ nats	$(0 + \log K + 0)/3$
Chance accuracy for y_q, y_{next}	$1/L = 0.20$	uniform guess
Accuracy without binding	$1/K + (1 - 1/K)/L = 0.60$	see text
Entropy of the emitted-symbol marginal	$\log N = 3.912$ nats	Proposition 1

3.3 Analytic reference values

Because the data-generating process is fully specified, the ideal value of every metric can be computed exactly (Table 1).

The no-binding baseline. A model that has learned "the answer is one of the context labels" but not which label belongs to which symbol can copy a context label at random. It is right when it copies the correct pair (probability $1/K$), and otherwise right by luck when the other pair happens to carry the same label (probability $1/L$). Its accuracy is

$$\frac{1}{K} + \Big(1 - \frac{1}{K}\Big)\frac{1}{L} = \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{5} = 0.60.$$

Accuracy above 0.60, not above 0.20, is therefore the evidence that the model performs in-context binding.

4 Model

4.1 Architecture

Both pipelines use the same causal, attention-only transformer, following the defaults of Singh et al. [4]: two layers, eight heads, model width $d = 64$ (head width 8), rotary position embeddings (RoPE [5]) on queries and keys, pre-layer normalisation, residual connections and no MLP blocks. With $e_t \in \{0, 1\}^{55}$ the one-hot input at position t:

$$h_t^{(0)} = W_{\text{in}}\, e_t + b_{\text{in}}, \tag{1}$$

$$h^{(l)} = h^{(l-1)} + \text{MHA}_l\big(\text{LN}_l(h^{(l-1)})\big), \qquad l = 1, 2, \tag{2}$$

$$z_t = W_{\text{out}}\, \text{LN}\big(h_t^{(2)}\big) + b_{\text{out}}, \tag{3}$$

where MHA_l is causal multi-head self-attention with a bias-free joint query–key–value projection and a bias-free output projection. The extended model imports the base model's blocks rather than copying them, so the two architectures are identical by construction. They differ only in the readout (Table 2):

- **base:** $W_{\text{out}} \in \mathbb{R}^{5\times 64}$, read at the query position $2K$;
- **extended:** $W_{\text{out}} \in \mathbb{R}^{55\times 64}$, read at positions $2K$, $2K+1$ and $2K+2$, each predicting the token that follows it. The logits are deliberately not restricted by token type, so emitting a symbol where a label belongs (or the reverse) is a real, measurable error.

Table 2: Parameter counts.

Component	Base	Extended
Input projection ($55 \to 64$, with bias)	3,584	3,584
Two attention blocks (each: LayerNorm 128, QKV 12,288, output 4,096)	33,024	33,024
Final LayerNorm	128	128
Output head	325	3,575
Total	37,061	40,311

4.2 Training objective

Both models are trained with Adam [6] at learning rate 10^{-4}, batch size 32 (sampled with replacement from the training pool) for 20,000 steps. The base model minimises the cross-entropy of y_q. The extended model is trained with teacher forcing: the true continuation sits in the input, all three positions are predicted in one forward pass, and the loss is the mean of the three per-token cross-entropies. The three per-token losses are also logged separately, because the mean is dominated by the irreducible $\log K$ term at x_{next}.

4.3 Generating continuations

When a trained model writes training data for the next generation:

- the **base** model's prediction is the argmax label, which is appropriate because its target is deterministic;
- the **extended** model generates autoregressively in three forward passes. At each step it reads the logits at the current position, chooses a token and writes that token's one-hot vector into the next input position before predicting the following token.

The rollout has to be autoregressive because y_{next} must be conditioned on the symbol the model actually emitted. Three decoding rules are available: `sample` (multinomial at temperature 1, used for every result here), `argmax`, and `hybrid` (argmax for y_q and y_{next}, sampling for x_{next}). Argmax would make the model always emit its favourite of the two context symbols, removing the coin flip from the data in a single generation.

5 Recursive training pipeline

1. **Fixed evaluation sets.** A validation set and a test set of 2,000 sequences each are drawn once from the true distribution and reused, unchanged, in every generation.
2. **Generation 0.** $\mathcal{D}^{(0)}$ holds 5,000 sequences from the true distribution, with true targets.
3. **Generation $i \geq 1$.** 5,000 fresh contexts are drawn from the true distribution, and their targets (base) or continuations (extended) are written by M_{i-1}. Contexts are always real, so the only way model error enters is through the generated tokens.
4. **Accumulation.** Model M_i trains on the pool $\mathcal{P}^{(i)} = \mathcal{D}^{(0)} \cup \cdots \cup \mathcal{D}^{(i)}$ of 5,000 $(i+1)$ sequences. Real data is never removed.

5. **Fresh start.** Every M_i is initialised from scratch, never warm-started from M_{i-1}.

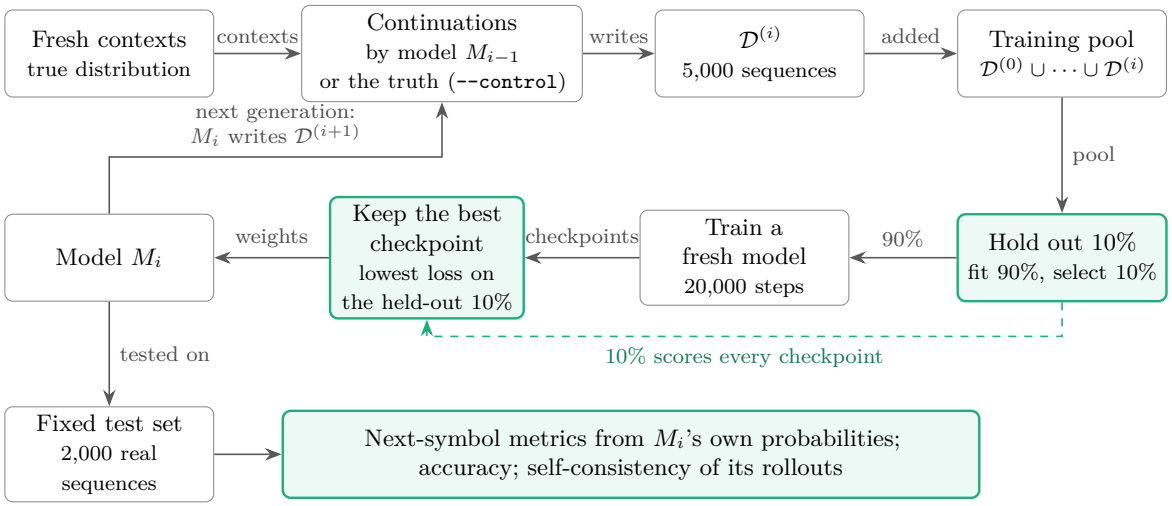


Figure 1: One generation of the extended pipeline. Green boxes are the revisions of Section 7. The base pipeline has the same loop with a single argmax target, no held-out selection and no control.

Table 3 summarises how the two pipelines differ as run for this report.

Table 3: Base and extended pipelines as run for this report. Task, architecture and optimiser settings are identical.

	Base	Extended
Tokens generated per sequence	1 (y_q)	3 (y_q, x_{next}, y_{next})
How generated tokens are chosen	argmax	sampled at temperature 1
Checkpoint kept each generation	last step	lowest loss on a held-out 10% of the pool
Control condition	none	matched real-data control
Seeds	1, 2, 3	1, 2, 3 in each condition
Collapse measures	test accuracy and loss; agreement of generated labels with the truth	per-token accuracy and loss; per-sequence next-symbol metrics; self-consistency; generated-data statistics
Script	`src/train_collapse.py`	`src/train_extended_collapse.py`

6 Measuring collapse

All model metrics are measured on the fixed test set, so a change across generations reflects the model, not a new sample of contexts.

6.1 Accuracy and loss per token

Per-token cross-entropy and argmax accuracy are computed with teacher forcing on the test set. Accuracy on y_q and y_{next} measures correctness; accuracy on x_{next} is capped at $1/K$ by construction and carries little information.

6.2 The next-symbol distribution

For test sequence n, let $C_n = \{c_{n,1}, \ldots, c_{n,K}\}$ be its context symbols and $p_n(\cdot)$ the model's softmax distribution at the x_{next} slot (conditioned on the true y_q). Define the in-context mass and the

renormalised choice

$$m_n = \sum_{j=1}^{K} p_n(c_{n,j}), \qquad \tilde{p}_{n,j} = \frac{p_n(c_{n,j})}{m_n}.$$

The four next-symbol metrics, each averaged over the $T = 2{,}000$ test sequences, are

$$\text{choice entropy} \quad \bar{H} = \frac{1}{T}\sum_n \Big(-\sum_j \tilde{p}_{n,j}\log\tilde{p}_{n,j}\Big), \tag{4}$$

$$\text{preferred-symbol probability} \quad \bar{\pi} = \frac{1}{T}\sum_n \max_j \tilde{p}_{n,j}, \tag{5}$$

$$\text{off-context probability} \quad \bar{o} = \frac{1}{T}\sum_n (1 - m_n), \tag{6}$$

$$\text{KL from truth} \quad \overline{\mathrm{KL}} = \frac{1}{T}\sum_n \mathrm{KL}(u_n \,\|\, p_n) = \frac{1}{T}\sum_n \Big(-\log K - \frac{1}{K}\sum_j \log p_n(c_{n,j})\Big), \tag{7}$$

where u_n is the true distribution, uniform on C_n. The choice entropy and preferred-symbol probability measure how decisively the model commits to one context symbol; the renormalisation stops leakage from inflating them. The off-context probability measures leakage to anything that was never in the context, wrong symbols and wrong-type tokens alike. The KL combines both failure modes; it equals the expected x_{next} test loss minus $\log K$, but is computed exactly instead of from one sampled target per sequence. Each metric was checked against hand-built distributions with known answers before use (uniform, deterministic, leaking and biased).

These quantities must be computed from probabilities. A generated dataset holds one sampled x_{next} per context, so no statistic of the data alone can show how sure the model was when it chose.

6.3 Self-consistency and type validity

The trained model generates continuations for the fixed test contexts. Among the sequences whose emitted $\hat{x}_n$ is a context symbol, self-consistency is the share whose emitted $\hat{y}_n$ equals that symbol's label in the context. It asks whether the model still binds symbol to label on its *own* generated symbol. A model whose choice has collapsed onto a favourite can still be perfectly self-consistent, whereas one whose binding has degraded cannot. It is a behavioural measure: attributing it to a specific circuit would require attention analysis or ablations. Type validity is the share of emitted tokens of the right kind for their slot (labels at positions 5 and 7, a symbol at position 6).

6.4 Statistics of the generated data

For each $\mathcal{D}^{(i)}$, the pipeline records the share of sequences whose x_{next} lies in the context, the agreement of each generated token with the continuation the true distribution would have produced for the same context, and the self-consistency of the generated data. Row i of these columns describes data written by M_{i-1}, not by M_i.

6.5 Why the emitted-symbol histogram cannot detect collapse

The original design measured collapse through the histogram of emitted x_{next} symbols over all N symbols: its entropy, coverage and top-1 mass. In this task that histogram cannot see per-sequence collapse.

Proposition 1. *Let the context symbols be drawn uniformly without replacement, and let a rule choose $x_{\text{next}} = c_\sigma$, where the slot $\sigma \in \{1, \ldots, K\}$ may depend on the slot structure of the sequence (for example the query index) but not on which symbols occupy the slots. Then $\Pr(x_{\text{next}} = s) = 1/N$ for every symbol s, whatever the rule.*

Proof. By exchangeability, the symbol in each slot is marginally uniform on $\{0, \ldots, N-1\}$, and this stays true conditional on the slot structure, which is drawn independently of the symbols. Choosing a slot without looking at the symbols therefore gives $\Pr(x_{\text{next}} = s) = \sum_k \Pr(\sigma = k) \Pr(c_k = s) = \sum_k \Pr(\sigma = k)/N = 1/N$. □

A rule that always picks the first context symbol is completely collapsed (choice entropy 0) yet produces a perfectly uniform histogram. A rule that looks at symbol identities, for example "always the smaller id", can skew the histogram, so the histogram remains useful as a check for *global* symbol bias. Section 7.2 shows the effect empirically.

7 Methodological revisions

The first version of the extended pipeline ran end to end and produced clean figures. A full five-generation run (seed 5, before any revision; per- generation results in Table 5) then turned up four problems, each serious enough to make a conclusion unreliable.

7.1 Why rigour matters for this project

Every claim this project makes is a comparison: extended against base, later generations against earlier ones, model-generated against real data. A comparison is only as trustworthy as the instrument that measures it and the controls around it. The first run's headline metric showed the true value, flat, in all five generations, which reads as "no collapse". That flat line turned out to be guaranteed by construction (Proposition 1). Written up as it stood, the conclusion would have been an artefact of the measurement, not a property of the model.

Synthetic tasks make rigour cheap: the true distribution is known analytically, every part of the data-generating process is under control, and a run takes half an hour on a laptop. The revisions therefore follow four rules:

1. **Check every metric against a known answer**, a healthy case and a collapsed one, before trusting what it says.
2. **Attribute an effect to recursive training only against a matched control** that changes nothing else.
3. **Replicate before interpreting**: report trends across several seeds, with every seed visible.
4. **Keep the truth out of the loop**: data used to evaluate the models never decides what the loop trains on or which checkpoint it keeps.

7.2 Revision 1: measure collapse where it can happen

Problem. The emitted-symbol histogram cannot detect per-sequence collapse (Section 6.5).

Evidence. Hand-built choices for 5,000 sequences were passed through the original metric (Table 4, Figure 2). The completely collapsed rules are indistinguishable from the truth. In the first run, the histogram stayed between 3.897 and 3.909 nats with 50/50 coverage in every generation, while the entropy of the model's own next-symbol distribution ranged from 0.337 to 0.712 nats (Table 5). The original claim that argmax decoding would "destroy the symbol distribution in a single generation" was wrong for the same reason: argmax would remove the coin flip from the data, but the histogram would not show it.

Change. Collapse is now measured from M_i's probabilities on the fixed test contexts (Section 6.2). The histogram is kept and relabelled as a global bias check.

Table 4: The emitted-symbol histogram under hand-built rules, 5,000 sequences. Fully collapsed rules score the same as the truth.

How x_{next} is chosen	Histogram entropy	Coverage	Choice entropy
Truth: coin flip between the two context symbols	3.908	50/50	0.693
Always the first context symbol	3.907	50/50	0
Always repeat the query symbol	3.907	50/50	0
Always the smaller symbol id	3.712	49/50	0

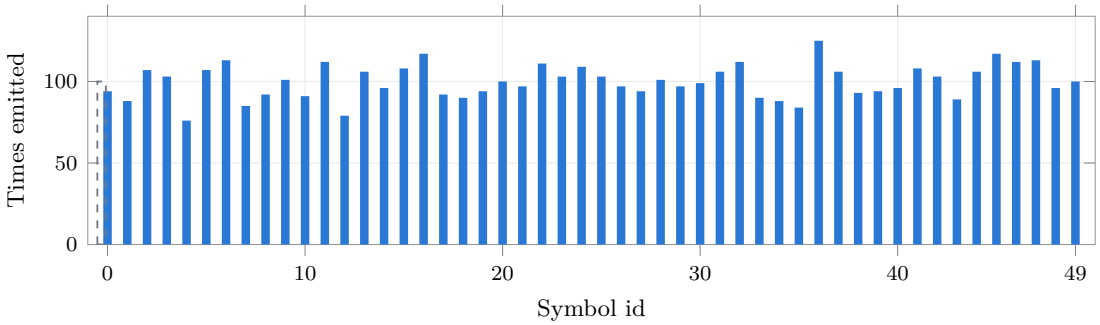


Figure 2: Emitted-symbol histogram of a completely collapsed rule (always pick the first context symbol) over 5,000 sequences. The dashed line is the uniform expectation of 100 per symbol. Entropy 3.907 nats, against 3.908 for the truth.

7.3 Revision 2: stop training before the model memorises its pool

Problem. 20,000 steps at batch size 32 over a 5,000-sequence pool is 128 passes over the data. Trained to the last step, M_0 memorised which symbol came next in each training sequence.

Evidence. Figure 3. From step 8,000 onwards, M_0's training loss was below the lowest value the coin flip allows (0.231), reaching 0.036 at step 15,000. Its next-symbol loss on held-out data bottomed out at 1.06 at step 5,000 and rose to 2.08 at step 20,000, three times its floor. On the test set, M_0 put 0.878 of its probability on its preferred context symbol (truth 0.5) and was over 90% sure in 63% of sequences. It was collapsed before any recursion happened, so every later generation would have been measured against a broken baseline.

Change. 10% of each generation's pool is held out uniformly at random. Every 500 steps the model is evaluated on it, and the checkpoint with the lowest held-out loss is kept (`select_holdout_fraction: 0.1`). The selection uses the pool rather than the fixed validation set because that set is real data. Choosing checkpoints with it would let the true distribution steer every generation of the synthetic loop, which can only hide collapse. The pool is all a model in the loop would have.

7.4 Revision 3: a matched real-data control

Problem. The pool grows every generation (from 5,000 to 25,000 sequences) while the training budget stays fixed, so "later generation" also means "more data, and fewer passes over it". Any trend across generations mixes the two.

Evidence. In the first run the entropy of the model's next-symbol distribution *rose* from 0.337 to 0.712 nats across generations (Table 5), the opposite of collapse and most likely the memorisation fading as the pool grew.

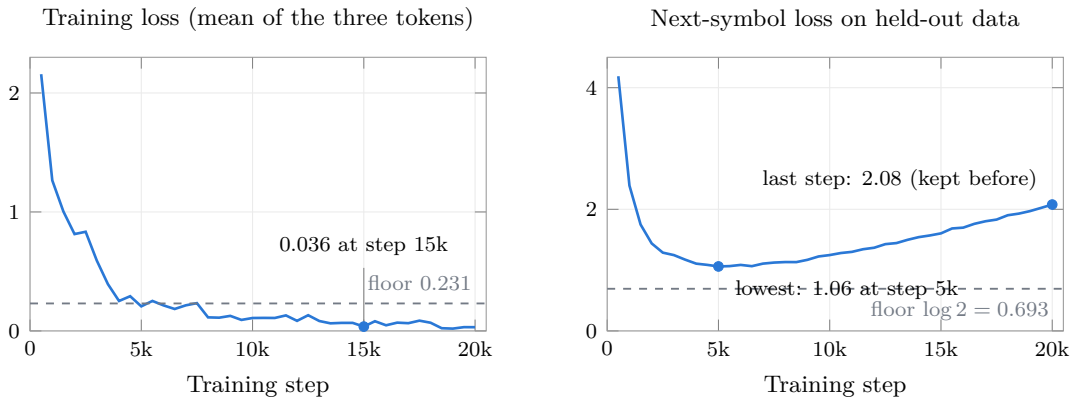


Figure 3: Generation 0 of the first (pre-revision) run, seed 5. Training loss fell below the lowest value the coin flip allows, while the next-symbol loss on held-out data rose after step 5,000. The original pipeline kept the last step; the revised one keeps the checkpoint with the lowest held-out loss.

Change. The `--control` flag reruns the identical pipeline with every generation's continuations drawn from the true distribution, so the pool grows to the same sizes but never holds model output (Figure 4). It is a flag on the same configuration file rather than a second file, so the two conditions cannot drift apart. With the same seed both conditions draw identical $\mathcal{D}^{(0)}$ and train identical M_0, so each seed is a paired comparison and only the gap between the conditions is attributed to training on model output.

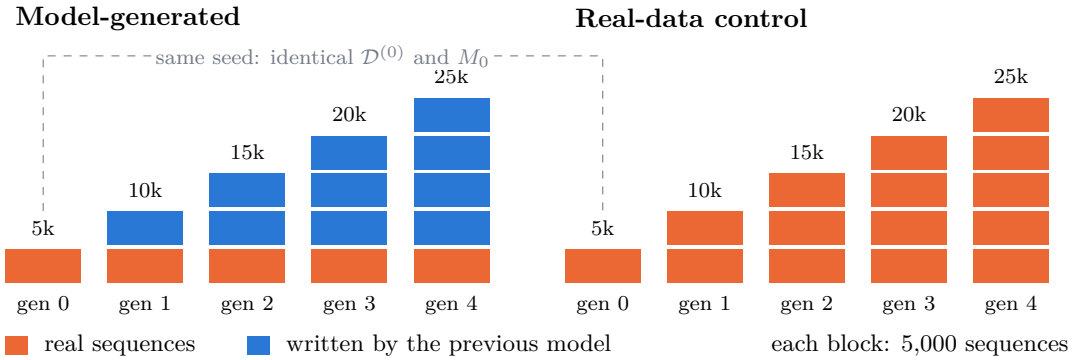


Figure 4: Training-pool composition by generation. Both pools grow by 5,000 sequences per generation, so anything the two runs share is the effect of more data; only the gap between them is the effect of training on model output.

7.5 Revision 4: replicate across seeds

Problem. The most interesting signal of the first run, the model's probability on symbols outside the context rising from 2.0% to 4.5%, came from a single seed. One run cannot separate an effect from that run's luck.

Change. `--seeds` runs independent replicates, each into its own directories, and `src/plot_extended_comparison.py` draws every seed alongside the mean. This report uses seeds 1, 2 and 3 in each condition.

7.6 Smaller corrections

- **One model per point.** The original summary figure put M_i's test accuracy next to statistics of $\mathcal{D}^{(i)}$, which M_{i-1} wrote, so two different models shared each generation. Single-run figures

now use only M_i's own test metrics.

- **The right accuracy baseline.** Accuracy plots show the 0.60 no-binding baseline next to chance at 0.20.
- **Behavioural, not mechanistic.** Self-consistency had been called a mechanistic probe; it is a behavioural one.

Table 5: The first full run, before the revisions (seed 5, trained to the last step, no control). Histogram columns describe $\mathcal{D}^{(i)}$ (written by M_{i-1}); the last three columns are M_i's own next-symbol distribution on the test set (full-vocabulary entropy, unrenormalised preferred-symbol probability, probability outside the context).

		Test accuracy / consistency			Histogram of $\mathcal{D}^{(i)}$		M_i's next-symbol distribution		
Gen	Pool	y_q	y_{next}	Self-cons.	Entropy	Coverage	Entropy	Preferred	Off-context
0	5,000	0.991	0.982	0.983	3.909	50	0.337	0.878	2.0%
1	10,000	0.992	0.986	0.985	3.900	50	0.503	0.822	3.0%
2	15,000	0.990	0.992	0.982	3.901	50	0.630	0.777	3.9%
3	20,000	0.995	0.993	0.980	3.897	50	0.663	0.770	4.0%
4	25,000	0.997	0.989	0.972	3.899	50	0.712	0.757	4.5%

8 Experimental setup

Table 6: Configuration (`config_collapse.yaml` for the base pipeline, `config_extended_collapse.yaml` for the extended one).

Setting	Base	Extended
Symbols N, labels L, context pairs K	50, 5, 2	50, 5, 2
Layers, heads, width	2, 8, 64	2, 8, 64
Optimiser, learning rate, batch size	Adam, 10^{-4}, 32	Adam, 10^{-4}, 32
Training steps per generation	20,000	20,000
Evaluation interval	500 steps	500 steps
Generations	5 (0 to 4)	5 (0 to 4)
New sequences per generation	5,000	5,000
Validation and test sets	2,000 each, fixed	2,000 each, fixed
Decoding of generated tokens	argmax	sampling, temperature 1
Held-out share for checkpoint selection	none	10%

Conditions. Three conditions, each with seeds 1, 2 and 3:

- **Extended, model-generated**: the extended pipeline with sampled continuations.
- **Extended, real-data control**: identical, but every generation's continuations come from the true distribution (`--control`).
- **Base**: the base pipeline, unchanged (argmax targets, last-step checkpoint, no control).

A seed fixes the data, the initialisation, the minibatch order and the sampled continuations, so the fixed validation and test sets also differ between seeds; each seed is an independent replicate.

Compute. All runs used a laptop CPU (Intel Core i7-1255U, 10 cores) with one thread per run, several runs in parallel. The six extended runs took 28 minutes of wall-clock time together; the three base runs took 14 minutes.

Reproduction. From the repository root:

```
python src/train_extended_collapse.py --config config_extended_collapse.yaml --seeds 1 2 3
python src/train_extended_collapse.py --config config_extended_collapse.yaml --control --seeds 1 2 3
python src/plot_extended_comparison.py "results/extended_collapse_metrics*_test_summary.csv"
python src/train_collapse.py --config config_collapse.yaml
```

The base script takes its seed and output paths from the configuration file; the base runs in this report used copies of `config_collapse.yaml` with `seed` set to 1, 2 and 3 and seed-specific output paths.

9 Results

9.1 Sanity checks

Pairing. Within each seed, the model-generated and control runs produced identical generation-0 results, to every logged digit, in all three seeds, as the paired design requires.

Checkpoint selection. Table 7 lists the step that selection kept. In generation 0 it stopped at step 5,500 in every seed, close to where the pre-revision run's held-out loss bottomed out (Figure 3). Later generations, with larger pools, stopped later.

Table 7: Training step of the kept checkpoint, model-generated / control.

Seed	Gen 0	Gen 1	Gen 2	Gen 3	Gen 4
1	5,500 / 5,500	10,500 / 15,000	12,000 / 12,500	11,000 / 12,000	17,500 / 17,000
2	5,500 / 5,500	8,000 / 7,500	17,500 / 11,000	17,000 / 12,000	17,000 / 16,000
3	5,500 / 5,500	8,500 / 6,500	10,000 / 14,000	14,500 / 16,500	18,500 / 15,500

9.2 Model-generated against real data

Figure 5 shows every seed of both conditions, and Tables 8 and 9 give the means. Generation 0 is shared. From generation 1 onwards the control improves on every measure as real data accumulates, while the model-generated chain barely moves.

Table 8: Accuracy and self-consistency on the fixed test set, mean over three seeds (model-generated / control).

	y_q accuracy		y_{next} accuracy		Self-consistency	
Gen	Model	Control	Model	Control	Model	Control
0	0.989	0.989	0.980	0.980	0.970	0.970
1	0.988	0.990	0.979	0.987	0.956	0.972
2	0.984	0.997	0.970	0.997	0.935	0.991
3	0.992	0.997	0.985	0.997	0.946	0.994
4	0.992	0.999	0.984	0.999	0.943	0.997

9.3 Paired differences

Table 10 gives the paired gap (model-generated minus control) at generation 4. In every metric the three seeds agree in sign, and the gap is between 2.7 and 12.9 times its seed-to-seed standard deviation. Table 11 shows that the gap grows with the generation for the next-symbol metrics, again with all three seeds agreeing in sign in every generation.

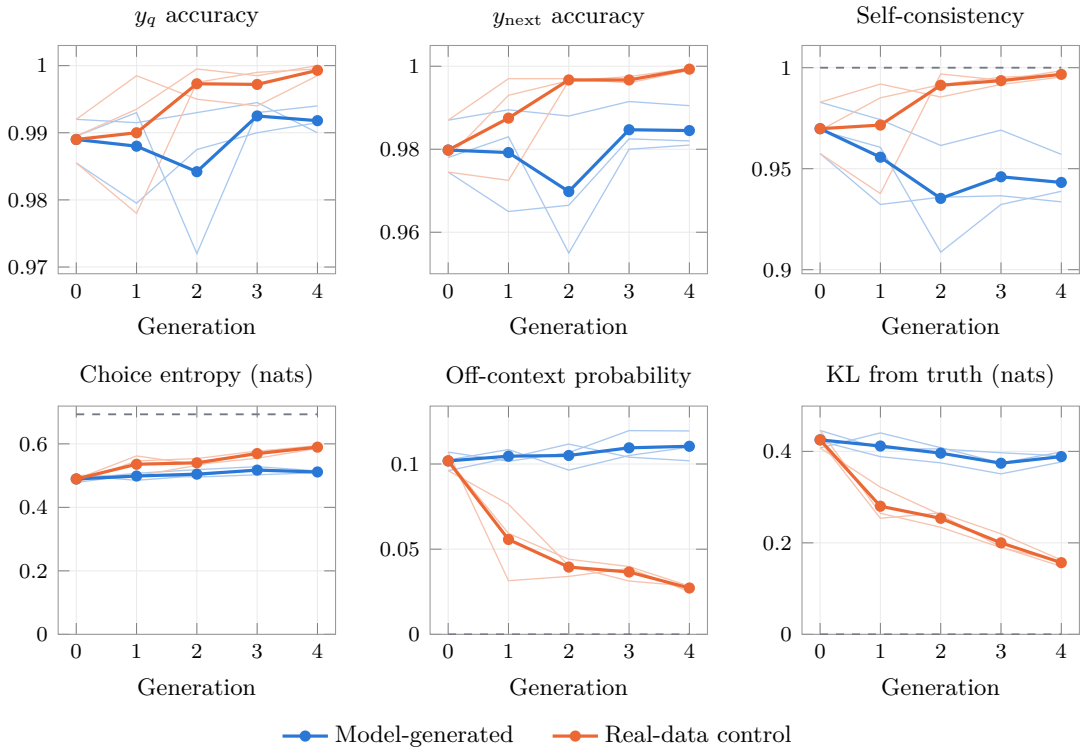


Figure 5: Extended pipeline, model-generated data (blue) against the matched real-data control (orange). Thin lines are individual seeds, thick lines the mean over the three seeds. Dashed lines are analytic reference values (Table 1); in the two accuracy panels the no-binding baseline of 0.60 lies below the plotted range.

Table 9: Next-symbol metrics on the fixed test set, mean over three seeds (model-generated / control). Truth: entropy 0.693, preferred 0.5, off-context 0, KL 0.

	Choice entropy		Preferred prob.		Off-context prob.		KL from truth	
Gen	Model	Control	Model	Control	Model	Control	Model	Control
0	0.489	0.489	0.760	0.760	0.102	0.102	0.425	0.425
1	0.499	0.536	0.753	0.727	0.105	0.056	0.412	0.280
2	0.505	0.540	0.750	0.723	0.105	0.039	0.396	0.254
3	0.517	0.569	0.741	0.700	0.110	0.037	0.374	0.200
4	0.511	0.590	0.745	0.682	0.110	0.027	0.389	0.157

Table 10: Paired gap at generation 4, model-generated minus control, over three seeds. "Gap/s.d." is the mean gap over the standard deviation of the three per-seed gaps. p is a two-sided paired t-test with 2 degrees of freedom; with three seeds such tests are fragile, and the consistency across seeds and generations is the stronger evidence.

Metric	Model	Control	Mean gap	Range over seeds	Gap/s.d.	p
KL from truth (nats)	0.389	0.157	+0.232	+0.214 to +0.250	12.9	0.002
Off-context probability	0.110	0.027	+0.083	+0.074 to +0.094	8.0	0.005
Choice entropy (nats)	0.511	0.590	−0.078	−0.084 to −0.071	11.9	0.002
Preferred-symbol probability	0.745	0.682	+0.063	+0.055 to +0.070	8.2	0.005
Self-consistency	0.943	0.997	−0.054	−0.065 to −0.038	3.9	0.021
y_{next} accuracy	0.984	0.999	−0.015	−0.018 to −0.009	2.7	0.043
y_q accuracy	0.992	0.999	−0.007	−0.008 to −0.006	5.7	0.010

Table 11: Mean paired gap (model-generated minus control) by generation. In every cell the three seeds agree in sign.

Gen	KL from truth	Off-context prob.	Choice entropy	Self-consistency
1	+0.132	+0.049	−0.037	−0.016
2	+0.142	+0.065	−0.036	−0.056
3	+0.174	+0.073	−0.052	−0.048
4	+0.232	+0.083	−0.078	−0.054

9.4 What the model-generated data contains

Table 12 describes the training data each model wrote. M_0 put about 10% of its next-symbol probability outside the context, so about 10% of the next symbols in $\mathcal{D}^{(1)}$ were symbols that never appeared in the sequence. The share then crept upwards as each model learned from data that already contained such errors.

Table 12: Generated data in the model-generated condition, three seeds: the share of sequences whose next symbol is outside the context (mean and range), the agreement of the generated y_q with the truth, and the data's self-consistency. Row i describes $\mathcal{D}^{(i)}$, written by M_{i-1}; row 0 is real data.

$\mathcal{D}^{(i)}$	Outside context	Range	y_q agreement	Self-consistency
0	0.0%	0.0–0.0%	1.000	1.000
1	10.0%	9.2–10.6%	0.975	0.968
2	10.7%	10.1–11.6%	0.967	0.961
3	10.9%	10.5–11.2%	0.955	0.935
4	11.6%	10.6–13.1%	0.966	0.950

9.5 The view without a control

Table 13 compares the model-generated chain with itself and with the control. Read on its own, the chain looks stable: KL from the truth even falls slightly, and the off-context probability rises by under one percentage point. That reading would conclude "no collapse". The control shows what the same growth in data does when the data is real: KL falls to 0.157 and leakage to 2.7%. The model-generated chain does not stay still so much as fail to improve, while its leakage and self-consistency drift slowly the wrong way.

Table 13: The model-generated chain against itself and against the control (means over three seeds).

Metric	Model-generated, gen 0	Model-generated, gen 4	Control, gen 4
KL from truth (nats)	0.425	0.389	0.157
Off-context probability	0.102	0.110	0.027
Choice entropy (nats)	0.489	0.511	0.590
Self-consistency	0.970	0.943	0.997

9.6 Base pipeline

Table 14 and Figure 6 give the base pipeline's results over the same three seeds. Test accuracy starts at 0.997 in generation 0 and ends at 0.999 in generation 4, and the targets each model wrote for the next generation agree with the truth in 99.72% to 99.96% of sequences. The base pipeline shows no sign of collapse. With at most 0.28% of its labels wrong, model-written data is almost indistinguishable from real data, and accuracy rises slightly as the pool grows, as it would with real data. Without a matched control the base pipeline cannot separate those two

explanations, but with labels this accurate the distinction hardly matters. The base model is also slightly more accurate than the extended model's y_q in either condition; the extended model learns three tokens and is stopped earlier by checkpoint selection, so that difference reflects the training setup, not recursion.

Table 14: Base pipeline, three seeds, with the extended pipeline's query-label results for comparison. Agreement in row i refers to the targets of $\mathcal{D}^{(i)}$, written by M_{i-1} (row 0 is real data).

	Base				Extended, model-generated	
Gen	Test acc.	Range	Test loss	Label agreement	y_q test acc.	y_q agreement
0	0.9973	0.9965–0.9980	0.0087	1.0000	0.9890	1.0000
1	0.9988	0.9985–0.9990	0.0043	0.9983	0.9880	0.9748
2	0.9985	0.9975–0.9995	0.0055	0.9992	0.9842	0.9671
3	0.9988	0.9985–0.9995	0.0055	0.9990	0.9925	0.9554
4	0.9993	0.9985–1.0000	0.0036	0.9995	0.9918	0.9659

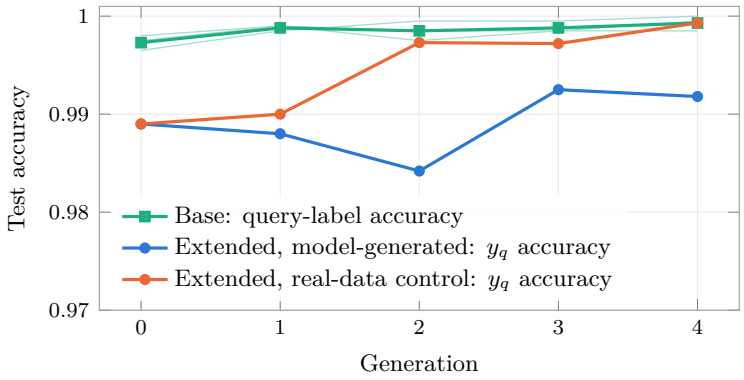


Figure 6: Query-label accuracy across generations: base pipeline (green; thin lines are seeds) and the extended pipeline's y_q in both conditions (means over three seeds).

10 Discussion

10.1 What the results show

Training on model output did not make the extended model collapse in the runaway sense: accuracy stayed above 0.97 and no metric deteriorated sharply. What it did was *lock in* the first model's errors. M_0, stopped at the best held-out loss, leaked about 10% of its next-symbol probability outside the context and was overconfident in its choice between the two context symbols (0.76 on its favourite, where the truth is 0.5). Sampling from M_0 wrote those errors into $\mathcal{D}^{(1)}$, and every later model learned them as part of the task. The real-data control, given the same growth in data, unlearned them: by generation 4 its leakage fell to 2.7% and its KL from the truth to 0.157 nats, against 11.0% and 0.389 nats in the model-generated chain.

This matches the accumulate regime of Gerstgrasser et al. [2]: because $\mathcal{D}^{(0)}$ stays in the pool, real data anchors every generation and the error stays bounded. Yet the anchor's weight shrinks from 100% of the pool to 20% by generation 4, and the gap to the control widens each generation, so the model-generated chain is not standing still relative to what the same amount of real data would achieve.

10.2 Mechanism

Two channels carry the errors forward. First, *leakage*: off-context symbols in the generated data teach the next model that the next symbol is sometimes not in the context at all, so its own leakage persists and slowly grows (10.0% of $\mathcal{D}^{(1)}$ to 11.6% of $\mathcal{D}^{(4)}$). Second, *overconfidence*: M_0's preferences between the two context symbols are a function of the context, so they are learnable. Sampled continuations encode those preferences as if they were real structure, and the next model imitates them, which keeps its choice entropy near 0.5 nats while the control's rises towards $\log 2$. Self-consistency falls because the generated data contains y_{next} labels that do not match their x_{next} (Table 12), weakening the symbol-to-label binding the third token depends on.

10.3 Base against extended

The base task has one deterministic output, so recursive training can only corrupt it through wrong labels, and its model makes almost none. Its model-written labels were wrong in at most 0.28% of sequences in any generation, so recursive training had almost nothing to propagate, and accuracy rose slightly, from 0.997 to 0.999, as the pool grew. The extended task exposes what accuracy cannot: its deterministic tokens stay at or above 0.97 accuracy in both conditions, but the distribution over the stochastic token, the leakage and the binding on the model's own outputs all separate the model-generated chain from the control. A collapse study that only measured accuracy would have reported no effect in either pipeline.

10.4 The role of checkpoint selection

Selection changed which errors M_0 passes on. Trained to the last step (first run), M_0 was strongly overconfident (choice entropy 0.337) but leaked little (2.0%); stopped at the best held-out loss, it is less overconfident (0.489) but leaks more (10.2%). The magnitude of every effect in this report therefore depends on M_0's error profile. A better M_0, trained on more real data or with regularisation, would pass on fewer errors. The direction of the effect, model-generated data preserving errors that real data removes, does not depend on this choice.

10.5 Limitations

- **Three seeds.** Every seed agrees in sign, but formal tests with three replicates are fragile.
- **Accumulation only.** The pool keeps all real data, which damps collapse by design. A replace regime would test the stronger form of collapse.
- **No rare-event tail.** The true next-symbol distribution is a fair coin, so the classic signature of collapse, rare outcomes disappearing [1], cannot occur in this task.
- **The base pipeline is not matched.** It keeps the last-step checkpoint and has no control, so base and extended differ in those respects as well as in the task.
- **One decoding rule and one task size.** Only sampling at temperature 1 and $K = 2$ were run.
- **Behavioural evidence.** No result here identifies an induction circuit or shows how it changes across generations.

10.6 Future work

- Run the replace regime alongside accumulation.
- Make the next-symbol distribution skewed (more context symbols with unequal probabilities) so that rare-but-valid continuations exist and their disappearance can be measured.

- Give the base pipeline the same checkpoint selection and control.
- Run the `argmax` and `hybrid` decoding contrasts.
- Track attention patterns and head ablations across generations to connect the behavioural effects to the induction circuit.

11 Conclusion

In a fully controlled in-context learning task, recursive training on model-generated data did not produce runaway collapse, as expected when real data accumulates. It did preserve the first model's errors, leakage outside the context, overconfident choices and weaker binding, which real data of the same quantity removed, and the gap widened every generation in all three seeds. Seeing this required measuring the next-symbol distribution through the model's own probabilities, stopping M_0 before it memorised its pool, and comparing against a matched real-data control. Without those, the same experiment would have reported "no collapse", for reasons that had nothing to do with the model.

A Repository map

File	Purpose
`src/data.py`, `src/model.py`	base task generator and the attention-only transformer
`src/collapse_data.py`, `src/train_collapse.py`	base recursive pipeline
`src/extended_data.py`	extended three-token task
`src/extended_model.py`	extended readout on the shared blocks
`src/extended_collapse_data.py`	autoregressive rollout and data generation
`src/extended_metrics.py`	collapse metrics, including `next_symbol_stats`
`src/train_extended_collapse.py`	extended pipeline: selection, `--control`, `--seeds`
`src/plot_extended_collapse_metrics.py`	figures for one run
`src/plot_extended_comparison.py`	comparison figure across conditions and seeds
`results/*_test_summary.csv`	per-generation summaries, one per run
`results/extended_comparison.png`	comparison figure (PNG version of Figure 5)

B Summary CSV columns (extended pipeline)

Column	Meaning
`seed`, `continuation_source`, `decode`	run identity (`model` or `real`; decoding rule)
`selected_step`	training step of the kept checkpoint
`test_loss_t1..t3`, `test_acc_t1..t3`	per-token loss and accuracy of M_i on the fixed test set
`test_next_context_entropy`	choice entropy $\bar{H}$
`test_next_preferred_prob`	preferred-symbol probability $\bar{\pi}$
`test_next_off_context_mass`	off-context probability $\bar{o}$
`test_next_kl_to_truth`	KL from the truth
`test_consistency`, `test_type_validity`	self-consistency and type validity of M_i's own rollouts
`data_*`	statistics of $\mathcal{D}^{(i)}$, written by M_{i-1} (row 0: real data)
`data_in_context_fraction`	share of $\mathcal{D}^{(i)}$ whose x_{next} is a context symbol
`data_symbol_*`	emitted-symbol histogram (global bias check only)

References

[1] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson and Y. Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024.

[2] M. Gerstgrasser, R. Schaeffer, A. Dey, et al. Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. arXiv:2404.01413, 2024.

[3] C. Olsson, N. Elhage, N. Nanda, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[4] A. K. Singh, T. Moskovitz, F. Hill, S. C. Y. Chan and A. M. Saxe. What needs to go right for an induction head? A mechanistic study of in-context learning circuits and their formation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021.

[6] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.